NY Vehicle Collision Analysis

Arushi Singhal (as4966) | Venkat Aditya Mullagiri (vm662) | Yug Patel (yp517)

GitHub Repo | Video Demo

..

1. Project Description

1.1. Quick Project Summary

- The goal of this project was to predict the accident severity based on various time-related factors. We analyzed a sample of 100k (out of more than 2 million) NYC motor vehicle collisions to identify any patterns related to the crashes. Using this, we assessed the accident severity to understand the impacts for further analysis.
- **Important Note**: Specific numbers in this report, such as severity scores and crash counts, may shift slightly between runs. This is because the data is pulled live from the NYC Open Data API, which is updated continuously. The patterns and conclusions remain consistent regardless of which sample is used.

1.2. Problem Statement

- How does the time of day impact the severity of an accident? Based on trends observed in the NYC motor vehicle collision data, what were the major contributing factors that influenced the number of injuries in a collision? Specifically looked into debunking common assumptions, like driving late at night or driving during the rush hour actually produce the most severe crashes, or do other factors like vehicle type, number of vehicles involved, and contributing behavior play a larger role? To answer these questions, the project analyzes 100,000 crash reports and builds a predictive model that identifies which factors are most influential in determining crash severity.

1.3. Connection to the Course (CS210)

- This project heavily relies on the concepts taught in the course Data Management for Data Science (CS210). The data was retrieved using the NYC Open Data API, demonstrating real-world API usage and data collection techniques. The data was then cleaned and pre-processed using the pandas library, including regex and feature engineering. A SQLite database was used to store the cleaned data, and a thorough exploratory analysis was conducted. The patterns observed using the SQL queries and the plots plotted were then used as a foundation for building the model, which was then improved slowly through trial and error, and the findings of the patterns found in the data exploratory analysis.

2. Novelty and Importance

2.1. Importance of the Project

- Traffic accidents are one of the leading causes of injury and death in urban areas, such as New York City. With its density, volume of vehicles, and mix of pedestrians, cyclists, and motorists, it is one of the most complex traffic environments in the country. While crash data is publicly available, it rarely gets explored in a way that goes beyond surface-level statistics. This project takes a deeper look by building a predictive model that identifies which combinations of factors, time of day, location,

vehicle type, and driver behavior are most likely to result in a severe crash. The findings could be used by the city planners to allocate emergency resources more effectively, traffic enforcement could be targeted at the highest-risk windows, and the model itself could eventually be integrated into a real-time alert system that helps get help to severe crashes faster.

2.2. Motivation and Relevance

- The motivation behind this project came from questioning assumptions that most people take for granted about road safety. Everyone assumes late-night driving is the most dangerous or that rush hour produces the worst crashes. But are those assumptions actually supported by the data? That question drove the entire direction of the project. Using a real dataset of 100,000 NYC crash reports, we were able to test these assumptions directly, and as the SQL analysis revealed, the results were often unexpected. Evening hours between 7pm and 10pm were actually more severe on average than late-night hours, and borough-level patterns showed that the Bronx at 10pm was the single most dangerous combination. This kind of data-driven insight is exactly what makes the project relevant. It replaces assumptions with evidence, which is eventually what good data science is supposed to do.

3. Progress

3.1. Retrieving the Data

- The dataset used in this project was sourced from the NYC Open Data portal, which provides publicly accessible data collected and maintained by the New York Police Department (NYPD). Rather than downloading the dataset, the Socrata Open Data API (SODA) was used to retrieve the data. This approach was used to ensure that any member of the project could just clone the repository without worrying about manually downloading data. Since the full dataset has over 2 million records, it is neither necessary nor practical. The API enforces a maximum limit of 50000 rows, so data was retrieved twice using Python's Request library to meet the 100k threshold set by the team. Both were ordered by crash date in descending order to prioritize the most recent records. The two records are merged and shuffled to finally make up the sample dataset used for the project. The shuffling was done with a fixed random seed to ensure reproducibility. The final sample of 100,000 rows was saved locally as a CSV file to serve as the input for the rest of the analysis and modeling.

3.2. Cleaning the Data

- Once the raw dataset was retrieved, the next step was to clean and prepare it for analysis. The raw data came with 29 columns, many of which were either irrelevant to our goal or too messy to use directly. The first thing we did was standardize all the column names. The API returned them in all caps with spaces, like "CRASH DATE" and "NUMBER OF PERSONS INJURED", so we converted everything to lowercase with underscores to make the data easier to work with. From there, we narrowed the dataset down to 19 columns that were actually relevant to our analysis, dropping things like street names, zip codes, and collision IDs that would not contribute anything meaningful to the model or the SQL queries.

The majority of the cleaning involved using regex to validate and fix messy values across several key columns. For crash time, we only kept values that matched a valid HH: MM format and set everything else to null, since malformed times would break the time-based features we were planning to engineer. For the borough column, we stripped out any digits or special characters, uppercased everything, and then checked whether the result matched one of the five valid NYC borough names. Anything that did not match was labeled "Unknown" rather than dropped, because those rows still had useful information like injury counts and vehicle types. We applied similar logic to the vehicle type column, which was a free-text field with dozens of variations for the same vehicle. Things like "4 door", "2 DOOR", and "Sedan" all mean the same thing. We used regex pattern matching to collapse all of these into eight clean categories like SEDAN, SUV, TRUCK, and BICYCLE. For the contributing factor column, we stripped out non-ASCII characters and extra whitespace that had crept in from the reporting system. Finally, for coordinates, we validated that latitude and longitude values fell within the geographic range of New York City, and nulled out anything that did not. Since a coordinate of 0.0 would place a crash in the middle of the ocean, and corrupt any location-based analysis.

After cleaning, we moved into feature engineering. From the date and time columns, we extracted the hour, month, and day of the week. We then built three binary flags: is_rush_hour (6–10 am and 4–8 pm), is_weekend (Saturday and Sunday), and is_night (midnight to 5 am) because these time windows represent genuinely different driving conditions and risk profiles that a raw hour number alone would not capture cleanly. We also created a factor_group column that collapsed the 50+ unique contributing factor descriptions into six broader categories: Distraction, Speeding, Alcohol/Drugs, Failure to Yield, Weather/Road, and Fatigue. This made the contributing factor usable for both the model and the SQL analysis, since having 50 distinct values would make it almost impossible to spot patterns. We counted the number of vehicles involved in each crash by checking how many of the five vehicle columns were non-null, since more vehicles typically means a more serious crash. Finally, we created the two most important columns for the rest of the project: severity_score, a numeric measure where each injury counts as 1 point and each fatality counts as 10, and is_severe, the binary target variable used in the model, which is set to 1 if a crash had 2 or more total injuries or at least one fatality. At the very end, we dropped any rows missing a crash time or date, since those two fields were the foundation of everything we were trying to analyze. The cleaned dataset was then saved as both a CSV file for modeling and plotting and loaded into a SQLite database.

3.3. SQL Query Exploration and Plots

- Before building any model, we needed to understand what the data was actually telling us. We ran 25 questions against the database using SQL and created 10 charts to explore the patterns. The goal was not just to understand the data but to use what we found to decide which factors the model should pay attention to.

The SQL analysis was done using 5 patterns: Time, Geographic, Contributing Factors, Vehicle type, and Combined Factor queries.

Time Queries:
These queries looked at crashes by hour, rush hour vs not, late night vs normal times, weekday vs weekend, and several other combinations. Plot 1 visually confirmed what query 1 found: the number of crashes peaks sharply during rush hour, making it look like the most dangerous time. But Plot 2 told a completely different story, showing severity climbing steadily through the evening and peaking at 9 pm, well after rush hour ends. This visual contrast between Plot 1 and Plot 2 was what first suggested that frequency and severity were two different things, i.e., frequency doesn't mean crashes were more severe. Query 6 showed another interesting pattern — that late night and weekend together was not the most severe combination. Plots 3, 4, and 5 all showed bars nearly identical in height, confirming that rush hour, weekend, and late night flags on their own were too weak to be useful alone. So we have to use these attributes in combination with something else.

Geographic Queries:
These queries looked at crash counts and severity by borough, by hour within each borough, and at late-night and weekend breakdowns by borough. Plot 6 showed Brooklyn had the most total crashes, which matched query 8. But query 9 showed Brooklyn at 8pm and 9pm had the highest average severity of any borough-hour combination at 0.718, with the Bronx close behind — showing that the Bronx produces dangerous crashes disproportionate to its total crash volume. These findings showed that location only matters in combination with time, so the model used a combination of location and time.

Contributing Factor Queries:
These were the most important queries for the model. Plot 7 showed that distraction causes far more crashes than any other factor. If the analysis had stopped at the chart, distraction would have seemed like the most important factor. But Plot 8 showed Failure to Yield had the highest severity when ranked by how bad crashes were, rather than how many there were. Query 12 showed that Failure to Yield had a severity score of 0.958 and a 69.4% chance of causing injury or death, and alcohol ranked last despite being assumed to be the deadliest. Query 13 showed alcohol crashes were most severe at 8 pm, not 2-3 am as expected. These findings led to adding a dedicated Failure to Yield flag as a model feature.

Vehicle Type Queries:
These queries looked at which vehicles were in the worst crashes and how the number of vehicles changed outcomes. Plot 9 supported query 18 that severity is directly proportional to the number of vehicles. This gave clear reason to add a high vehicle count flag to the model. Q17 showed bicycles had an 86.6% chance of injury in a crash, mopeds 84.4%, and motorcycles 69.0%, compared to 39% for regular cars. Plot 10 supported this finding that cyclist injuries peaked at 4-6 pm, the same evening window

that the time queries found to be most severe overall. Query 17 directly became the vulnerable vehicle flag in the model, grouping bikes, mopeds, and motorcycles together because their injury rates were so much higher than everything else.

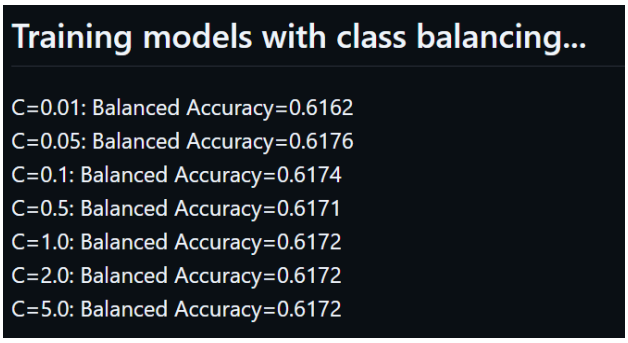
Combined Factor Queries:
These queries looked at combining multiple factors at the same time, for example, alcohol at night by borough, motorcycle crashes by hour, and speeding by borough and time of day. These queries were explored to support building the model. Query 23 showed Queens had the most late-night alcohol crashes, ahead of Brooklyn and Manhattan, which was not visible from either the borough chart or the factor chart alone. Query 24 showed motorcycle crashes peaking in the afternoon and evening, which, combined with Query 17, meant motorcycles were both common and dangerous during the exact hours identified as most severe. Query 25 showed that Brooklyn during the evening rush had the most speeding crashes of any combination. These queries confirmed the model needed features that combined multiple factors rather than looking at each one separately.

The SQL findings directly shaped the modeling phase. Three features were added to the model specifically because of what the queries revealed: an evening rush with multiple vehicles flag based on the severity spike, a dedicated Failure to Yield binary flag, and a vulnerable vehicle flag covering bicycles, motorcycles, and mopeds.

3.4. Models, Techniques, and Algorithms

- There was a lot of trial and error involved with the modeling portion of the project. The main idea was to run the features through various models and compare the accuracies to see which performed the best. The models we were assessing were logistic regression, random forest, gradient boosting, and AdaBoost. We started by defining the crash severity prediction as a binary classification, as the target variable “is_severe” was labeled as a boolean for whether crashes resulted in 2+ injuries or any fatalities. This helped to balance out the dataset, as 58.1% were labeled as non-severe, while 41.9% were severe. Through multiple iterations of improving the model, we felt the need to include more feature engineering, such as converting time into a cyclical transformation, rush hour indicators, whether it’s a weekend, and seasonal indicators for the winter. We also decided to assess the historical crash severity for the boroughs to generate a risk score. There were also more features regarding weather conditions, alcohol involvement, and so on. However, what helped us increase the model’s accuracy the most was integrating 3 of the most significant findings from the SQL queries. With this, we included the features of dangerous combos (those in the evening hours with more vehicles), accidents where there was a failure to yield (shown to be highest in severity), and accidents involving vulnerable vehicles (bikes, mopeds, and motorcycles). Over the iterations of testing, we realized that to help handle the class imbalance, we needed to add class weighting parameters *class_weight='balanced'* for Logistic Regression and Random Forest, while computing balanced sample weights for Gradient Boosting. After noticing that the logistic regression model had the best

performance, we decided to fine-tune that model further. Exploring the effects of the different regularization strengths, it was determined that *C=0.05* resulted in the highest balanced accuracy. Below, you can see the balanced accuracies with the different C-values.



For context, in the beginning, we had only been tracking the *Accuracy* and the *AUC-ROC*. However, the realization that we needed to balance the data sparked the idea to not only include a *Balanced Accuracy* metric, but also use that as our main evaluation metric. This was an important change for us to make, as the standard accuracy was extremely misleading for our imbalanced dataset. For example, during one of the iterations, AdaBoost had achieved a 0.767 accuracy (matched the baseline) but only achieved a 0.505 balanced accuracy (showing that the model didn't learn anything about the data).

- ➢ Another key thing we worked on was the preprocessing pipeline, which consisted of a stratified 80/20 train-test split. We also had target encoding fitted exclusively on the training data to prevent leakage, computed missing values with median imputation, and handled outliers for feature scaling with a RobustScalar (performed better than the StandardScalar). Handling these critical precautions against data leakage also included computing all of the risk scores on just the training data, fitting all the preprocessors on the training before transforming the test data, and also only performing one test-train split.

3.5. Final Results and Evaluation

- ➢ By the end of this project, our best model was the logistic regression, which showed a balanced accuracy of 61.76% and AUC-ROC of 66.41. Even though these numbers are relatively low, this model represents a 23.5 % improvement over random guessing since a completely random model would only score 50%.

Model	Accuracy	Balanced Accuracy	AUC-ROC
Logistic Regression	0.63190	0.617598	0.664070
Gradient Boosting	0.63810	0.614864	0.673883
Random Forest	0.63755	0.606710	0.652992
AdaBoost	0.65380	0.601423	0.658456

BEST MODEL: Logistic Regression
Balanced Accuracy: 0.6176
AUC-ROC: 0.6641

- Comparing these results to a baseline model, which simply predicts "non-severe" for every crash, would catch 0% of the severe crashes, even though it would appear 58% accurate. Our model shows meaningful improvement from this, as we are able to successfully catch almost 70% of the severe crashes (instead of zero).
- Given that the dataset provided us with a limited number of external factors, such as weather data or traffic volume or traffic history, all valid and key details necessary to make an accurate prediction, our model shows significant performance.

4. Future Work

4.1. Potential Improvements

- As mentioned earlier, the lack of certain key details played a role in our success. It we were able to enhance it with more external factors like the weather data and traffic volume data, the model would definitely see some improvement in predicting severe crashes more accurately. Another bit of information that may prove to be important is information about the driver (age, previous driving history, license status, etc.). With this information, we might be able to identify other key patterns or combinations of circumstances that play a role in the outcome of an accident.
- Another major improvement that we could make is by deploying the model as a real-time prediction system. Being fed the new information, it could play a role in detecting emergency situations sooner, and get help to the area sooner. This could end up saving people's lives, as certain situations have the potential of being addressed before it becomes too critical.

4.2. Branching Project Ideas

- One potential sector for this project could be assessing the severity of vehicular collisions in other major cities. With this, not only can we understand the patterns behind accidents in different cities, but we can also make a comparison of which cities tend to have more severe crashes, more frequent accidents, and so on. There are numerous possibilities of how the project can be extended. This may include understanding the social patterns in different cities, which may cause higher chances of an accident (e.g., more traffic in warmer cities during "vacation periods" due to an increase in tourists).

Note: Each group member will be submitting a separate write-up of their individual contribution(s).